NavaCity Smart City

Interface, data and authority review - change report

SIMULATED DATA. NavaCity is a fictional city. Every route, station, passenger count, crowd level and forecast in this report comes from a rule-based simulator and a model trained on that simulated data. Nothing here describes a real city, operator, passenger survey or real-world validation.

Date	21 October 2026
Project	NavaCity Smart City - AI-Based Public Transport Demand & Overcrowding Prediction
Context	VTU B.E. Artificial Intelligence & Machine Learning mini-project
Stack	FastAPI, scikit-learn, SQLite, React 18, Vite 5, Tailwind CSS 3, Recharts 2
Scope of the review	20 reported issues (7 critical, 10 important, 3 optional) + a full redesign of the public and the authority interfa
Data	95,568 simulated observations, 2025-01-01 to 2025-06-30, 4 per route-stretch per day
Repository	No git repository is initialised in the working directory, so the changes are delivered as files rather than as co

1. Summary of the review

The review covered the whole product: the public journey planner, the transport-authority dashboard, the backend API, the simulated dataset and the documentation. The baseline was already **fully green** - 112 unit/API tests, 61 setup checks, the live-API script and the frontend wiring checks all passed - so none of the reported issues was a regression. Every one of them was a gap in behaviour, labelling, documentation or layout that the existing test suite never asserted.

All 20 reported issues were traced to a concrete file and line. 19 were confirmed exactly as described; one (C5, the route map) had a different cause than reported but a real defect. None was found to be unreproducible.

Severity	Issues	Confirmed	Partly true	Files changed	New tests
Critical	**7**	**6**	**1**	**9**	**13**
Important	10	10	0	8	7
Optional	3	3	0	6	0
Total	20	19	1	21	20

1.1 The three findings that mattered most

- **The origin and the destination never reached the model.** The forecast always used the busiest stretch of the line, so changing "Travelling from" changed nothing. The forecast is now made for the stretch of the journey the traveller actually takes, and the ride time covers the whole journey.

- **The data was described two different ways.** The README said "4 observations per segment per day", the dashboard said "95,568 simulated hourly observations". Both described the same rows, only one described the granularity correctly. The dashboard now reports 42 bus stops (the places a bus can stop at) instead of 5 (the bus-only stops), and the granularity is stated in the README, in the KPI text and in every chart caption.

- **A silent SQL bug.** func.sum() on a Boolean column returns the boolean True on SQLite instead of a count, so every "share of overcrowded observations" was wrong. The column is now cast to an integer first, and a regression test pins the number down.

1.2 The test results

Check	Command	Result
Unit and API tests	python -m pytest backend\tests -q	PASS - 131 passed, 1 skipped (baseline: 112 passed)
Setup verification	python verify_setup.py	PASS - 74/74 checks (baseline: 61/61)
Live API check	python backend\test_live_api.py	PASS - every check, including the 401 without a token
Frontend wiring	python frontend_check.py	PASS - 26/26 checks (baseline: 13/13)
Production build	npm run build	PASS - 905 modules transformed, no error
Manual smoke test	backend and frontend running	PASS - metro, bus, mode switch, alternative search, out-of-range date, s

2. Every issue, its status and what changed

The full log - with the root cause and the file and line of every issue - is in **docs/issue_log.md**. The table below is the summary.

ID	Severity	Status	What changed	Files affected
C1	Critical	confirmed	The origin and the destination never reached the model: the forecast always used the busiest stretch of the line, and the alternatives panel showed a title that did not match its rows.	navacity_core/network.py; backend/app/services/model_service.py; backend/app/schemas.py; backend/app/api.py; backend/app/services/alternatives_service.py; frontend/src/pages/PublicDashboard.jsx
C2	Critical	confirmed	The default date (2026-10-22) was outside the training window, the picker had no min/max and nothing warned about an extrapolation.	backend/app/api.py; backend/app/schemas.py; navacity_core/config.py; frontend/src/pages/PublicDashboard.jsx; README.md
C3	Critical	confirmed	The README said "4 observations per segment per day", the dashboard said "95,568 simulated hourly observations". The dashboard also counted 5 bus stops instead of the 42 places a bus can stop at.	README.md; backend/app/services/analytics_service.py; backend/app/database.py; backend/app/api.py; verify_setup.py
C4	Critical	confirmed	Two charts on the public page used different units (passengers vs percent) and different scopes, and neither had an axis unit.	frontend/src/pages/PublicDashboard.jsx; frontend/src/components/ChartCard.jsx
C5	Critical	partly true	The map mixed three colour systems (per-line metro colours, a single bus amber and the crowd palette), so the selected route could be painted in the wrong colour and the legend described neither.	frontend/src/components/TransitMap.jsx; frontend/src/styles/tokens.css; frontend/tailwind.config.js
C6	Critical	confirmed	The global header showed "Authority signed in" and a "Sign out" button on the public page.	frontend/src/App.jsx; frontend/src/pages/AuthorityDashboard.jsx
C7	Critical	confirmed	.gitignore did not list .env and .env.example shipped an active demo password.	.gitignore; .env.example; verify_setup.py
I1	Important	confirmed	"Journey time" was the travel time of a single stretch.	backend/app/services/model_service.py; backend/app/schemas.py; frontend/src/components/AnswerCard.jsx
I2	Important	confirmed	The alternatives were ranked by crowd level and wait, never by the total journey time, and carried no total time at all.	backend/app/services/alternatives_service.py; backend/app/schemas.py; frontend/src/components/AlternativeCard.jsx
I3	Important	confirmed	The dashboard reported 5 bus stops (the bus-only stops) and 41 metro stations; the network has 42 places a bus can stop at.	backend/app/services/analytics_service.py; backend/app/api.py; backend/tests/test_journey.py

ID	Severity	Status	What changed	Files affected
I4	Important	confirmed	The purple dashed "total" series had no human name, and several charts had no axis unit and no data-period note.	frontend/src/components/ChartCard.jsx; frontend/src/pages/AuthorityDashboard.jsx
I5	Important	confirmed	The map had no visible station labels, only a tooltip.	frontend/src/components/TransitMap.jsx
I6	Important	confirmed	The bus/metro comparison used absolute passengers per segment-hour, although a metro seats 320 and a bus 48.	backend/app/schemas.py; backend/app/services/analytics_service.py; backend/app/api.py; frontend/src/pages/AuthorityDashboard.jsx
I7	Important	confirmed	The waiting-time formula was undocumented and untested.	backend/app/services/model_service.py; README.md; backend/tests/test_journey.py
I8	Important	confirmed	The README login example showed issued_at equal to expires_at.	README.md
I9	Important	confirmed	The public page showed the internal model name "gradient_boosting".	frontend/src/pages/PublicDashboard.jsx
I10	Important	confirmed	The month features were not covered for months 7-12 and nothing said so.	navacity_core/config.py; backend/app/api.py; README.md; backend/tests/test_journey.py
O1	Optional	confirmed	The number formatter was duplicated in two files.	frontend/src/lib/format.js; frontend/src/pages/PublicDashboard.jsx; frontend/src/pages/AuthorityDashboard.jsx
O2	Optional	confirmed	The README had a duplicated section 7.2, a second section 8, a missing section 9 heading, a wrong package.json path and an ambiguous test-file path.	README.md
O3	Optional	confirmed	The public page was a flat column of controls, cards, two charts, a map and an explainer.	frontend/src/pages/PublicDashboard.jsx; frontend/src/components/*

3. Before and after: the public interface

3.1 The structure

	Before	After
Top of the page	A title, a date line, then a filter card with five controls and a run button.	The answer card: the crowd level as a large badge, one plain se
Controls	Always expanded, in one flat grid. The mode toggle ran on every change.	A collapsed summary bar once a result exists ("Metro - M1 Blue Line - Ty
The journey	"Travelling from" and "Travelling to" sat in a second grid below the rest, with a swap button.	A From / To picker with a swap button, filtered to the stations the chosen
Results	Four cards: predicted demand, waiting time, journey time and crowd level.	One answer card. The model name, the raw metrics and the internal arg
Charts	Two charts with different units: passengers per hour and minutes.	One 24-hour chart, "Expected crowd by hour", in passengers per hour, w
The map	A full-height map with three colour systems and no collapse.	A collapsible section. One colour system (metro solid indigo, bus dashed
The explainer	Always visible, as a long list of colour dots.	A collapsible "How crowd levels are defined" section in plain language, w
The header	Showed "Authority signed in" and a "Sign out" button.	The product name, a "Simulated data" pill, the navigation and the API sta

3.2 The measured effect of the journey fix

The forecast for **M1 Blue Line** at 08:00 on 2025-06-30, before and after the fix (the numbers come from the live API):

Journey	Stretch forecast	Ride time	Total journey
NavaCity Central → Mahatma Nagar (2 stops)	M1-S01	5.7 min	9.6 min
NavaCity Central → Whitefield Depot (7 stops)	M1-S06	34.2 min	36.1 min

Before the fix both journeys returned the same stretch (M1-S06) and the same 5.7 min ride time, so the "Travelling from" control had no effect at all.

4. Before and after: the authority interface

The authority area is the transport planner's tool. The goal of the redesign was to support a service-planning decision, separately for the metro and the bus, without the planner having to read the whole page first.

4.1 The sections

Section	Before	After
Navigation	No navigation. Everything was on one long page: KPIs, charts, tables, alerts and recommendations.	A sticky left sidebar with the four sections: Overview, Metro, Bus, Model &
Overview	Six KPI cards, one chart of metro against bus, and both modes mixed in the tables.	Six KPI cards (routes, metro stations, bus stops, simulated passengers, a
Metro and Bus	Did not exist. The single page mixed both modes.	The same page, filtered to one mode, served by the same endpoint with
The route table	A plain table of total passengers, mean occupancy, overcrowded share and peak hour.	The same table, sortable by mean occupancy, overcrowded share and pe
The alerts	A table with the mode, the route, the segment id, the severity and the sketch.	The same information, with the segment as two station names, the severity
The recommendations	A table of route, hour, current and recommended frequency, and the status.	A card per recommendation that states the current frequency, the sugges
Model & Data	The model name and the raw metrics, mixed into the overview.	Its own section: the metrics per mode with the test period and the sample
The header	The global header, shared with the public page.	Its own header: "Signed in as transport authority", a Sign out button and t

Every number on the authority dashboard comes from one endpoint and one function. There is no hard-coded figure in the React code: the KPIs, the route table, the alerts and the recommendations are read from /api/authority/stats, /api/alerts, /api/crowd-summary and /api/authority/mode-summary.

5. The design system

The goal was a calm, credible transport product rather than a dashboard full of decoration. The rules live in **docs/design_system.md**; the tokens live in **frontend/src/styles/tokens.css** and are mirrored in **frontend/tailwind.config.js**.

5.1 The token table

Token	Value	Used for
metro	#1D3FBF (indigo-700)	metro lines, primary actions, the selected nav item
metro-soft	#EEF2FF (indigo-50)	tinted metro backgrounds
bus	#B7791F (amber-600)	bus routes, the bus family
bus-soft	#FFFBEB (amber-50)	tinted bus backgrounds
crowd low	#0F8A5F (emerald-600)	the Low level, filled-circle icon
crowd moderate	#0369A1 (sky-600)	the Moderate level, half-filled-circle icon
crowd high	#C2410C (orange-600)	the High level, triangle icon
crowd critical	#B91C1C (red-700)	the Critical level, octagon icon
heading / body / muted	#0F172A / #334155 / #64748B	slate-900 / 700 / 500
border / page / surface	#E2E8F0 / #F8FAFC / #FFFFFF	slate-200 / 50, white
notice	amber-50 / amber-900 / amber-200	the simulated-data banner
focus	#4F46E5 (indigo-600)	the 2 px focus ring with a 2 px offset
radius	12 / 8 / 999 px	cards / inputs and buttons / badges
spacing	4 px grid	card padding 20 px (public) and 16 px (authority)
content width	1152 px	the centred maximum width
type scale	12 / 14 / 16 / 18 / 24 / 32 px	16 px body on the public page, 14 px on dense tables

5.2 The component list

Component	What it guarantees
CrowdBadge	an icon shape and a text label, never colour alone
OccupancyBar	the occupancy bar with the 40 / 65 / 85 % threshold ticks
ModeSwitch	the Metro / Bus segmented control, arrow-key support, aria-pressed
StationPicker	the From / To pickers, filtered to the route, with a swap button
AnswerCard	the single primary answer: badge, one sentence, wait / ride / total, bar
AlternativeCard	route, mode, total time, crowd badge, transfers, the difference
StatCard	a KPI card with a label, a value, a unit and a context line
ChartCard / LegendItem	a title, a visible legend, the plot and a caption with the period
DataNotice	the simulated-data banner, identical on every page
EmptyState / ErrorState	a short message plus a retry action, never a blank panel
LoadingSkeleton	skeletons shaped like the content, never a spinner

CollapsibleSection	a foldable section with an aria-expanded button
TransitMap	one colour system, visible station labels, a matching legend
lib/format.js	the one number formatter (Indian grouping)

5.3 Screenshots

No browser-automation tool (Playwright or Selenium) is installed in this environment, so screenshots of the running application could not be captured automatically. The before/after of each screen is therefore described in §3 and §4 instead, and the acceptance criteria were verified by reading the built CSS and by driving the live API and the live frontend.

6. What changed in the backend

The existing endpoints all keep working; the changes are additive. A new endpoint was added only where the frontend needed it.

Endpoint	Change
POST /api/predict	accepts origin_id and destination_id; answers with the journey, the ride time of the whole journey
GET /api/predict	the same, as query parameters.
GET /api/predict/day	the same, for the 24-hour chart of the chosen journey.
GET /api/alternatives	answers with the total journey time of every alternative, the total journey time of the route that wa
GET /api/authority/stats	accepts mode=metro|bus. Answers with the data period, the granularity, the weekday/weekend p
GET /api/authority/alerts	accepts mode=metro|bus.
GET /api/authority/recommendations	accepts mode=metro|bus and states the rule that produced every recommendation.
GET /api/authority/mode-summary	new - the side-by-side comparison of the two modes with the comparable metrics and the

6.1 The security was not weakened

The authority endpoints still answer **401 Unauthorized** without a valid session token, the token is still held in memory only in the browser, the demo password is still never returned by any endpoint and it is still refused when NAVACITY_ENV=production. The live check confirms the 401, and the sign-out still invalidates the token immediately. The only change to the authentication surface is that the **session state moved out of the public header** into the authority area - the protection itself is unchanged.

7. Known limitations that remain

- **All the data is simulated.** NavaCity is a fictional city. The model is trained on a rule-based simulator, so the reported metrics describe how well the model fits that simulator. They say nothing about how the system would perform on real passenger counts.

- **Dates outside the simulated window are extrapolations.** The model was trained on 2025-01-01 to 2025-06-30. The date picker is limited to that window, and any forecast for a day outside it is marked as an extrapolation. The month features were only observed for January to June, so a forecast for July to December extrapolates a seasonal pattern that was never observed. The model was **not** retrained, because no test showed a defect in the data or in the features.

- **The data is a sample, not a complete hourly series.** Each route-stretch has 4 sampled hours per day (the two commuter peaks plus two others). The 24-hour chart interpolates between them, and a mean over a stretch-hour is an average over 181 days of four samples.

- **The frequency recommendations are a heuristic.** The rule is "add one vehicle per hour above 80 % occupancy, remove one below 40 %". It is not an optimised schedule and it ignores the cost of the vehicles, the depot capacity and the driver roster. A planner has to validate every change.

- **The journey resolution uses the busiest stretch of the journey.** The model forecasts one stretch at a time, so the forecast for a journey is the forecast for the busiest stretch the traveller passes. The ride time, on the other hand, is the sum of every stretch of the journey.

- **The model is rebuilt from scratch each time.** The model bundle and the metrics file are artefacts, not part of the source. Re-running ml/train_model.py reproduces them from the same seed.

- **No screenshots of the running application.** No browser-automation tool is installed in this environment, so the visual acceptance criteria were verified by inspecting the built CSS and by driving the live application through its API, not by capturing the screens.

- **No git commits.** The working directory is not a git repository, so the changes are delivered as files rather than as a commit history.

8. The files that were added

File	What it is
docs/baseline_report.md	the measured baseline, the facts taken from the data and the status of every reported issue
docs/issue_log.md	every issue with its status, root cause, fix and files
docs/design_system.md	the tokens, the components and the layout rules
docs/change_report.pdf	this report
backend/tests/test_journey.py	20 new tests: the journey, the date range, the alternatives, the wait formula, the stop count and th
frontend/src/styles/tokens.css	the design tokens
frontend/src/lib/format.js	the one number formatter
frontend/src/components/AnswerCard.jsx	the primary answer of the public interface
frontend/src/components/AlternativeCard.jsx	one less crowded alternative
frontend/src/components/ModeSwitch.jsx	the Metro / Bus switch
frontend/src/components/StationPicker.jsx	the From / To pickers with a swap button
frontend/src/components/ChartCard.jsx	the chart wrapper with the legend and the caption
frontend/src/components/StatCard.jsx	the KPI card
frontend/src/components/DataNotice.jsx	the simulated-data banner
frontend/src/components/CollapsibleSection.jsx	the foldable section